NOSQL DATABASE DESIGN – LAB SHEET 3

MongoDB Shell (mongosh) | Questions, Code, Queries and Expected Outputs

QUESTION 1

```
lab.students.drop();
lab.students.insertMany([
  {name:"Aman", age:20, courses:["Python","MongoDB"], attendance:90,
   skills:["Coding","Communication"], activities:["Quiz","Assignment"]},
  {name:"Riya", age:21, courses:["Java","Python"], attendance:65,
   skills:["Design"], activities:["Quiz"]},
  {name:"Kabir", age:20, courses:["MongoDB"], attendance:82,
   skills:["Coding"], activities:["Project"]}
]);
show("Query 1: Students with attendance above 80", lab.students.find({attendance:{$gt:80}}, {_id:0}));
show("Query 2: Students enrolled in Python", lab.students.find({courses:"Python"}, {_id:0}));
show("Query 3: Students who did an Assignment", lab.students.find({activities:"Assignment"}, {_id:0}));
/*
EXPECTED OUTPUT (key fields):
Query 1: Aman (90), Kabir (82)
Query 2: Aman, Riya
Query 3: Aman
*/
```

QUESTION 2

```
lab.events.drop();
lab.events.insertMany([
  {user:"Aman", event:"login"},
  {user:"Aman", event:"failed-login"},
  {user:"Aman", event:"failed-login"},
  {user:"Riya", event:"file-upload"},
  {user:"Riya", event:"logout"},
  {user:"Aman", event:"password-change"},
  {user:"Riya", event:"login"}
]);
show("Query 1: Find failed-login events", lab.events.find({event:"failed-login"}, {_id:0}));
show("Query 2: Users with at least 2 failed logins", lab.events.aggregate([
  {$match:{event:"failed-login"}},
  {$group:{_id:"$user", attempts:{$sum:1}}},
  {$match:{attempts:{$gte:2}}}
]));
show("Query 3: Count each event type", lab.events.aggregate([
  {$group:{_id:"$event", count:{$sum:1}}},
  {$sort:{_id:1}}
]));
/* EXPECTED OUTPUT: Aman has 2 failed logins; counts: failed-login 2, login 2,
   file-upload 1, logout 1, password-change 1. */
```

QUESTION 3

```
lab.blogs.drop();
lab.blogs.insertMany([
  {title:"Learn Python", author:"Aman", category:"Technology",
   tags:["Python","Coding"], comments:[{user:"Riya",text:"Great blog!"}],
   published:true, publishedAt:new Date("2026-01-10")},
  {title:"Healthy Living", author:"Riya", category:"Lifestyle",
   tags:["Health","Fitness"], comments:[], published:true,
   publishedAt:new Date("2026-02-05")},
  {title:"MongoDB Tips", author:"Kabir", category:"Technology",
   tags:["MongoDB","Coding"], comments:[], published:false}
]);
show("Query 1: Blogs in Technology category", lab.blogs.find({category:"Technology"}, {_id:0}));
show("Query 2: Blogs tagged Python", lab.blogs.find({tags:"Python"}, {_id:0}));
show("Query 3: Blogs containing BOTH Python and Coding tags",
  lab.blogs.find({tags:{$all:["Python","Coding"]}}, {_id:0}));
show("Query 4: Published blogs", lab.blogs.find({published:true}, {_id:0}));
/* EXPECTED OUTPUT: Technology -> Learn Python, MongoDB Tips;
   Python -> Learn Python; both tags -> Learn Python; published -> Learn Python, Healthy Living. */
```

QUESTION 4

```
lab.products.drop();
lab.products.insertMany([
  {name:"Dell Laptop", category:"Laptop", RAM:"16GB", storage:"512GB SSD", price:60000},
  {name:"Samsung Mobile", category:"Mobile", battery:"5000mAh", camera:"50MP", price:25000},
  {name:"Canon Camera", category:"Camera", resolution:"24MP", lens:"18-55mm", price:45000}
]);
show("Query 1: Laptops with 16GB RAM", lab.products.find({category:"Laptop",RAM:"16GB"}, {_id:0}));
show("Query 2: Mobiles with 50MP camera", lab.products.find({category:"Mobile",camera:"50MP"}, {_id:0}));
show("Query 3: Cameras with 24MP resolution", lab.products.find({category:"Camera",resolution:"24MP"}, {_id:0}));
/* EXPECTED OUTPUT: Dell Laptop; Samsung Mobile; Canon Camera, respectively. */
```

QUESTION 5

```
lab.orders.drop();
lab.orders.insertMany([
  {customerId:101, items:[{product:"Laptop",price:50000,quantity:1},{product:"Mouse",price:500,quantity:2}]},
  {customerId:102, items:[{product:"Mobile",price:20000,quantity:1}]},
  {customerId:101, items:[{product:"Keyboard",price:1500,quantity:1}]}
]);
show("Query 1: Complete purchase history for customer 101", lab.orders.find({customerId:101}, {_id:0}));
show("Query 2: Calculate each order total", lab.orders.aggregate([
  {$unwind:"$items"},
  {$group:{_id:"$_id", customerId:{$first:"$customerId"},
    total:{$sum:{$multiply:["$items.price","$items.quantity"]}}}},
  {$sort:{customerId:1}}
]));
/* EXPECTED OUTPUT: Customer 101 order totals: 51000 and 1500; customer 102: 20000.
   Order _id values are generated by MongoDB and vary. */
```

QUESTION 6

```
lab.visits.drop();
lab.visits.insertMany([
  {page:"/home", userId:1, device:"Mobile", browser:"Chrome", location:"Delhi", time:new Date()},
  {page:"/home", userId:2, device:"Desktop", browser:"Edge", location:"Mumbai", time:new Date()},
  {page:"/products", userId:3, device:"Mobile", browser:"Chrome", location:"Delhi", time:new Date()},
  {page:"/home", userId:4, device:"Mobile", browser:"Firefox", location:"Pune", time:new Date()}
]);
show("Query 1: Most visited pages", lab.visits.aggregate([
  {$group:{_id:"$page", visits:{$sum:1}}}, {$sort:{visits:-1,_id:1}}
]));
show("Query 2: Commonly used devices", lab.visits.aggregate([
  {$group:{_id:"$device", count:{$sum:1}}}, {$sort:{count:-1,_id:1}}
]));
/* EXPECTED OUTPUT: /home = 3 visits, /products = 1; Mobile = 3, Desktop = 1. */
```

QUESTION 7

```
lab.articles.drop();
lab.articles.insertMany([
  {title:"MongoDB Basics", author:"Aman", category:"Education",
   tags:["Database","NoSQL"], status:"published"},
  {title:"Java Introduction", author:"Riya", category:"Education",
   tags:["Java"], status:"draft"},
  {title:"AI News", author:"Kabir", category:"Technology",
   tags:["AI"], status:"unpublished"}
]);
show("Query 1: Published articles", lab.articles.find({status:"published"}, {_id:0}));
show("Query 2: Unpublished/draft content", lab.articles.find({status:{$ne:"published"}}, {_id:0}));
show("Query 3: Education articles tagged Java", lab.articles.find({category:"Education",tags:"Java"}, {_id:0}));
/* EXPECTED OUTPUT: Published -> MongoDB Basics; unpublished/draft -> Java Introduction, AI News;
   Education + Java -> Java Introduction. */
```

QUESTION 8

```
lab.carts.drop();
lab.carts.insertOne({userId:101, items:[
  {product:"Pen",price:10,quantity:2},
  {product:"Book",price:100,quantity:1}
]});
show("Initial cart", lab.carts.find({userId:101}, {_id:0}));
show("Query 1: Add Pencil", lab.carts.updateOne(
```

```
  {userId:101}, {$push:{items:{product:"Pencil",price:5,quantity:3}}}
));
show("Query 2: Change Book quantity to 2", lab.carts.updateOne(
  {userId:101,"items.product":"Book"}, {$set:{"items.$.quantity":2}}
));
show("Query 3: Remove Pen", lab.carts.updateOne(
  {userId:101}, {$pull:{items:{product:"Pen"}}}
));
show("Cart after changes", lab.carts.find({userId:101}, {_id:0}));
show("Query 4: Calculate cart value", lab.carts.aggregate([
  {$match:{userId:101}}, {$unwind:"$items"},
  {$group:{_id:"$userId", total:{$sum:{$multiply:["$items.price","$items.quantity"]}}}}
]));
/* EXPECTED OUTPUT: final items Book (100 x 2) and Pencil (5 x 3); total = 215. */
```

QUESTION 9

```
lab.reviews.drop();
lab.reviews.insertMany([
  {product:"Laptop", customer:"Aman", rating:5, comment:"Excellent"},
  {product:"Laptop", customer:"Riya", rating:4, comment:"Very good"},
  {product:"Mobile", customer:"Aman", rating:3, comment:"Average"}
]);
show("Query 1: Average rating and number of reviews per product", lab.reviews.aggregate([
  {$group:{_id:"$product", averageRating:{$avg:"$rating"}, totalReviews:{$sum:1}}},
  {$sort:{_id:1}}
]));
/* EXPECTED OUTPUT: Laptop averageRating=4.5, totalReviews=2;
   Mobile averageRating=3, totalReviews=1. */
```

QUESTION 10

```
lab.inventory.drop();
lab.inventory.insertMany([
  {product:"Laptop", stock:5, minStock:10, supplier:"ABC Traders", restockDate:new Date("2026-10-20")},
  {product:"Mouse", stock:30, minStock:10, supplier:"XYZ Traders", restockDate:new Date("2026-10-25")},
  {product:"Keyboard", stock:10, minStock:10, supplier:"Key Supplies", restockDate:new Date("2026-10-18")}
]);
show("Query 1: Products at or below minimum stock", lab.inventory.find({
  $expr:{$lte:["$stock","$minStock"]}
}, {_id:0}));
show("Query 2: Products with zero stock", lab.inventory.find({stock:0}, {_id:0}));
show("Query 3: Restock Laptop to 25 units", lab.inventory.updateOne(
  {product:"Laptop"}, {$set:{stock:25}}
));
/* EXPECTED OUTPUT: Laptop (5 <= 10) and Keyboard (10 <= 10) need restocking;
   zero-stock query returns no documents; update modifies 1 document. */
```

QUESTION 11

```
print("REQUIREMENT: This question must run on a replica set or sharded cluster.");
print("The transaction section is guarded so the rest of the file can still run on a standalone server.");
try {
  const session = db.getMongo().startSession();
  const bank = session.getDatabase("LabSheet3");
  try {
    session.startTransaction();
    bank.wallets.deleteMany({user:{$in:["Aman","Riya"]}});
    bank.wallets.insertMany([
      {user:"Aman",balance:5000},
      {user:"Riya",balance:2000}
    ]);
    const debit = bank.wallets.updateOne(
      {user:"Aman",balance:{$gte:1000}}, {$inc:{balance:-1000}}
    );
    if (debit.matchedCount !== 1) throw new Error("Insufficient balance or sender not found");
    const credit = bank.wallets.updateOne(
      {user:"Riya"}, {$inc:{balance:1000}}
    );
    if (credit.matchedCount !== 1) throw new Error("Receiver not found");
    session.commitTransaction();
    print("Transfer successful");
    show("Wallet balances after transfer", bank.wallets.find(
      {user:{$in:["Aman","Riya"]}}, {_id:0}
```

```
    ));
  } catch (transactionError) {
    try { session.abortTransaction(); } catch (ignore) {}
    print("Transaction failed: " + transactionError.message);
    print("Check that MongoDB is running as a replica set or sharded cluster.");
  } finally {
    session.endSession();
  }
} catch (setupError) {
  print("Transaction could not start: " + setupError.message);
  print("A standalone MongoDB server does not support multi-document transactions.");
}
/* EXPECTED OUTPUT on a replica set:
   Transfer successful
   { user: "Aman", balance: 4000 }
   { user: "Riya", balance: 3000 } */
```

QUESTION 12

```
lab.notifications.drop();
lab.notifications.insertMany([
  {userId:101, message:"Order shipped", type:"order", read:false},
  {userId:101, message:"Welcome", type:"system", read:true},
  {userId:101, message:"New message", type:"message", read:false}
]);
show("Query 1: Retrieve unread notifications", lab.notifications.find(
  {userId:101,read:false}, {_id:0}
));
show("Query 2: Mark 'Order shipped' as read", lab.notifications.updateOne(
  {userId:101,message:"Order shipped"}, {$set:{read:true}}
));
show("Notifications after update", lab.notifications.find({userId:101}, {_id:0}));
/* EXPECTED OUTPUT: initially Order shipped and New message are unread;
   update modifies one document; Order shipped becomes read=true. */
```

QUESTION 13

```
lab.learning.drop();
lab.learning.insertMany([
  {student:"Aman", course:"MongoDB", instructor:"Mr. Sharma",
   modules:["Basics","Queries","Aggregation"], progress:80, completed:false},
  {student:"Riya", course:"Python", instructor:"Ms. Gupta",
   modules:["Basics","Functions"], progress:100, completed:true},
  {student:"Kabir", course:"MongoDB", instructor:"Mr. Sharma",
   modules:["Basics"], progress:40, completed:false}
]);
show("Query 1: Students with progress above 70%", lab.learning.find({progress:{$gt:70}}, {_id:0}));
show("Query 2: Students who completed the course", lab.learning.find({completed:true}, {_id:0}));
show("Query 3: Students learning MongoDB", lab.learning.find({course:"MongoDB"}, {_id:0}));
/* EXPECTED OUTPUT: progress >70 -> Aman and Riya; completed -> Riya;
   MongoDB course -> Aman and Kabir. */
```

QUESTION 14

```
lab.tracking.drop();
lab.tracking.insertOne({
  orderId:1001, currentStatus:"Shipped",
  history:[
    {status:"Placed",time:new Date("2026-10-10T09:00:00Z")},
    {status:"Confirmed",time:new Date("2026-10-10T10:00:00Z")},
    {status:"Shipped",time:new Date("2026-10-11T12:00:00Z")}
  ]
});
show("Query 1: Retrieve current order status", lab.tracking.find(
  {orderId:1001}, {_id:0,orderId:1,currentStatus:1}
));
show("Query 2: Update status to Delivered and append history", lab.tracking.updateOne(
  {orderId:1001},
  {$set:{currentStatus:"Delivered"},
   $push:{history:{status:"Delivered",time:new Date("2026-10-12T15:00:00Z")}}}
));
show("Query 3: Retrieve complete order history", lab.tracking.find({orderId:1001}, {_id:0}));
/* EXPECTED OUTPUT: currentStatus initially Shipped; after update currentStatus=Delivered;
   history contains Placed, Confirmed, Shipped, Delivered. */
```

QUESTION 15

```
lab.news.drop();
lab.news.insertMany([
  {title:"Technology Summit", author:"Aman", category:"Technology",
   location:"Delhi", tags:["AI","Technology"], images:["summit.jpg"], source:"News Daily"},
  {title:"City Rainfall Update", category:"Weather", location:"Mumbai", source:"City News"},
  {title:"AI Innovation", author:"Riya", category:"Technology", tags:["AI","Research"]}
]);
show("Query 1: Technology articles", lab.news.find({category:"Technology"}, {_id:0}));
show("Query 2: Articles tagged AI", lab.news.find({tags:"AI"}, {_id:0}));
show("Query 3: Articles where author field exists", lab.news.find({author:{$exists:true}}, {_id:0}));
show("Query 4: Articles without an images field", lab.news.find({images:{$exists:false}}, {_id:0}));
show("Query 5: Search using different optional fields", lab.news.find({
  $or:[{category:"Technology"},{tags:"AI"},{location:"Delhi"}]
}, {_id:0}));
/* EXPECTED OUTPUT: Technology -> Technology Summit, AI Innovation;
   AI tag -> Technology Summit, AI Innovation; author exists -> Technology Summit, AI Innovation;
   images missing -> City Rainfall Update, AI Innovation;
   OR query -> all three articles. */
print("\nLAB SHEET 3 COMPLETED.");
```